

</>

</>

↩ ↗

```
1  @{\n2  ViewData["Title"] = "Employee Management";\n3  }\n4  \n5  \n6  <head>\n7  <title> @ViewData["Title"] - Admin Dashboard</title>\n8  \n9  </head>\n10 \n11 \n12 <div class="container mt-4">\n13 <h2> Employee Management</h2>\n14 \n15 \n16 \n17 <a asp-controller="Employee" asp-action="Create" class="btn btn-primary mb-3">Add New Employee</a>\n18 \n19 <table class="table table-bordered table-striped">\n20 <thead class="thead-dark">\n21 <tr>\n22 <th>ID</th>\n23 <th>Name</th>\n24 <th>Email</th>\n25 <th>Phone</th>\n26 <th>Address</th>\n27 </tr>\n28 </thead>\n29 <tbody>\n30 <tr>\n31 <td>1</td>\n32 <td>John Doe</td>\n33 <td>john.doe@example.com</td>\n34 <td>1234567890</td>\n35 <td>123 Main St, New York, NY 10001</td>\n36 </tr>\n37 </tbody>\n38 </table>\n39 \n40 </div>\n41 \n42 </div>
```

</>

1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23

```
using Microsoft.AspNetCore.Mvc;
using YourNamespace.Models;
using System.Linq;

namespace YourNamespace.Controllers
{
    public class EmployeeController : Controller
    {
        private readonly ApplicationDbContext _context;

        public EmployeeController(ApplicationDbContext context)
        {
            _context = context;
        }
    }
}
```

EmployeeController

Id

UserName

Email

Password

ConfirmPassword

RegisterViewModel

EmployeeController

Employee

using

Employee

Employee

ETMS.Models

csharp

```
using System.ComponentModel.DataAnnotations;

namespace ETMS.Models
{
    public class Employee
    {
        [Key]
        public int Id { get; set; }

        [Required]
        public string UserName { get; set; }

        [Required]
```

```
[EmailAddress]
public string Email { get; set; }

[Required]
public string Password { get; set; }

[Required]
public string ConfirmPassword { get; set; }

[Required]
public string Role { get; set; }
}
}
```

Employee

ETMS.Models

using

EmployeeManagementController

```
csharp

using ETMS.Models;
```

ApplicationDbContext

DbSet<Employee>

ApplicationDbContext

ETMS.Data

DbSet<Employee>

```
csharp

using ETMS.Models;
using Microsoft.EntityFrameworkCore;

namespace ETMS.Data
{
    public class ApplicationDbContext : DbContext
    {
        public ApplicationDbContext(DbContextOptions<ApplicationDbContext> options) : base(options)
        {
        }

        public DbSet<Employee> Employees { get; set; }
    }
}
```

```
bash

dotnet build
```

RegisterViewModel

EmployeeManagementController

Employee

- RegisterViewModel
- Employee
- Employee
- Employee ETMS.Models

Employee

Employee

csharp

```
using System.ComponentModel.DataAnnotations;

namespace ETMS.Models
{
    public class Employee
    {
        [Key]
        public int Id { get; set; }

        [Required(ErrorMessage = "User Name is required.")]
        [StringLength(255, ErrorMessage = "User Name must be at most 255 characters long.")]
        public string UserName { get; set; }

        [Required(ErrorMessage = "Email is required.")]
        [EmailAddress(ErrorMessage = "Invalid email format.")]
        public string Email { get; set; }

        [Required(ErrorMessage = "Password is required.")]
        [StringLength(100, MinimumLength = 6, ErrorMessage = "Password must be between 6 and 100 characters.")]
        public string Password { get; set; }

        [Required(ErrorMessage = "Confirm Password is required.")]
        [Compare("Password", ErrorMessage = "Passwords do not match.")]
        public string ConfirmPassword { get; set; }

        [Required(ErrorMessage = "Role is required.")]
        [RegularExpression("Admin|Trainer|Employee", ErrorMessage = "Invalid role selection.")]
        public string Role { get; set; }
    }
}
```

Employee

ETMS.Models

ApplicationDbContext

```
csharp

using ETMS.Models;
using Microsoft.EntityFrameworkCore;

namespace ETMS.Data
{
    public class ApplicationDbContext : DbContext
    {
        public ApplicationDbContext(DbContextOptions<ApplicationDbContext> options) : base(options) { }

        public DbSet<Employee> Employees { get; set; }
    }
}
```

using

```
csharp

using ETMS.Models;
```

```
bash

dotnet ef migrations add AddEmployeeModel
dotnet ef database update
```

RegisterViewModel

Employee

RegisterViewModel

EmployeeManagementController

RegisterViewModel

Employee

EmployeeManagementController

```
csharp

using Microsoft.AspNetCore.Mvc;
using ETMS.Models;
using System.Linq;
using ETMS.Data;

namespace ETMS.Controllers
{
    public class EmployeeManagementController : Controller
    {
        private readonly ApplicationDbContext _context;
```

```
public EmployeeManagementController(ApplicationDbContext context)
{
    _context = context;
}
```

// GET: Employee

```
public IActionResult Index()
{
    var employees = _context.Employees.ToList();
    return View(employees);
}
```

// GET: Employee/Create

```
public IActionResult Create()
{
    return View();
}
```

// POST: Employee/Create

```
[HttpPost]
[ValidateAntiForgeryToken]
public IActionResult Create(RegisterViewModel model)
{
    if (ModelState.IsValid)
    {
        _context.Employees.Add(new RegisterViewModel
        {
            UserName = model.UserName,
            Email = model.Email,
            Password = model.Password,
            ConfirmPassword = model.ConfirmPassword,
            Role = model.Role
        });
        _context.SaveChanges();
        return RedirectToAction(nameof(Index));
    }
    return View(model);
}
```

// GET: Employee/Edit/5

```
public IActionResult Edit(int id)
{
    var employee = _context.Employees.Find(id);
    if (employee == null)
    {
        return NotFound();
    }
    return View(employee);
}
```

// POST: Employee/Edit/5

```
[HttpPost]
[ValidateAntiForgeryToken]
public IActionResult Edit(int id, RegisterViewModel model)
{
    if (id != model.Id)
    {
        return NotFound();
    }

    if (ModelState.IsValid)
    {

```

```
var employee = _context.Employees.Find(id);
if (employee == null) return NotFound();

employee.UserName = model.UserName;
employee.Email = model.Email;
employee.Password = model.Password;
employee.ConfirmPassword = model.ConfirmPassword;
employee.Role = model.Role;

_context.Employees.Update(employee);
_context.SaveChanges();
return RedirectToAction(nameof(Index));
}
return View(model);
}

// POST: Employee/Delete/5
[HttpPost]
[ValidateAntiForgeryToken]
public IActionResult Delete(int id)
{
    var employee = _context.Employees.Find(id);
    if (employee == null)
    {
        return NotFound();
    }

    _context.Employees.Remove(employee);
    _context.SaveChanges();
    return RedirectToAction(nameof(Index));
}

// GET: Employee/Details/5
public IActionResult Details(int id)
{
    var employee = _context.Employees.Find(id);
    if (employee == null)
    {
        return NotFound();
    }
    return View(employee);
}
}
```

RegisterViewModel

ApplicationDbContext

Employees

csharp

```
public DbSet<RegisterViewModel> Employees { get; set; }
```

```
bash

dotnet ef migrations add AddEmployeeTable
dotnet ef database update
```

RegisterViewModel

Id

Id

RegisterViewModel

RegisterViewModel

Id

RegisterViewModel

```
csharp

using System.ComponentModel.DataAnnotations;

namespace ETMS.Models
{
    public class RegisterViewModel
    {
        [Key] // This marks Id as the primary key for EF
        public int Id { get; set; }

        [Required(ErrorMessage = "User Name is required.")]
        [StringLength(255, ErrorMessage = "User Name must be at most 255 characters long.")]
        public string UserName { get; set; }

        [Required(ErrorMessage = "Email is required.")]
        [EmailAddress(ErrorMessage = "Invalid email format.")]
        public string Email { get; set; }

        [Required(ErrorMessage = "Password is required.")]
        [StringLength(100, ErrorMessage = "Password must be at least {2} characters long.", MinimumLength = 6)]
        public string Password { get; set; }

        [Required(ErrorMessage = "Confirm Password is required.")]
        [Compare("Password", ErrorMessage = "Passwords do not match.")]
        public string ConfirmPassword { get; set; }

        [Required(ErrorMessage = "Role is required.")]
        [RegularExpression("Admin|Trainer|Employee", ErrorMessage = "Invalid role selection.")]
        public string Role { get; set; }
    }
}
```


dotnet ef migrations add AddEmployeeId

bash

dotnet ef database update

Employee

RegisterViewModel

Employee

Employee

csharp

```
using System.ComponentModel.DataAnnotations;

namespace ETMS.Models
{
    public class Employee
    {
        [Key]
        public int Id { get; set; }

        [Required]
        public string UserName { get; set; }

        [Required]
        public string Email { get; set; }

        [Required]
        public string Password { get; set; }

        [Required]
        public string ConfirmPassword { get; set; }

        [Required]
        public string Role { get; set; }
    }
}
```

ApplicationDbContext

csharp

```
using ETMS.Models;
using Microsoft.EntityFrameworkCore;

namespace ETMS.Data
{
    public class ApplicationDbContext : DbContext
    {
        public ApplicationDbContext(DbContextOptions<ApplicationDbContext> options) : base(options) { }
    }
}
```

```
public DbSet<Employee> Employees { get; set; }  
}  
}
```

```
bash  
  
dotnet ef migrations add CreateEmployeeTable  
dotnet ef database update
```



```
csharp  
  
using Microsoft.AspNetCore.Identity;  
using System.ComponentModel.DataAnnotations;  
  
namespace ETMS.Models  
{  
    public class ApplicationUser : IdentityUser  
    {  
        [Required(ErrorMessage = "User Name is required.")]  
        [StringLength(255, ErrorMessage = "User Name must be at most 255 characters long.")]  
        public string UserName { get; set; }  
  
        [Required(ErrorMessage = "Password is required.")]  
        public string Password { get; set; }  
  
        [Required(ErrorMessage = "Confirm Password is required.")]  
        [Compare("Password", ErrorMessage = "Passwords do not match.")]  
        public string ConfirmPassword { get; set; }  
  
        [Required(ErrorMessage = "Role is required.")]  
        public string Role { get; set; }  
    }  
}
```

ApplicationDbContext

ApplicationDbContext

ApplicationUser

```
csharp

using ETMS.Models;
using Microsoft.AspNetCore.Identity.EntityFrameworkCore;
using Microsoft.EntityFrameworkCore;

namespace ETMS.Data
{
    public class ApplicationDbContext : IdentityDbContext<ApplicationUser>
    {
        public ApplicationDbContext(DbContextOptions<ApplicationDbContext> options) : base(options) { }
    }
}
```

EmployeeManagementController

ApplicationUser

```
csharp

using ETMS.Models;
using ETMS.Data;
using Microsoft.AspNetCore.Mvc;
using Microsoft.AspNetCore.Identity;
using System.Linq;
using System.Threading.Tasks;

namespace ETMS.Controllers
{
    public class EmployeeManagementController : Controller
    {
        private readonly UserManager<ApplicationUser> _userManager;

        public EmployeeManagementController(UserManager<ApplicationUser> userManager)
        {
            _userManager = userManager;
        }

        // Display Employees
        public IActionResult Index()
        {
            var employees = _userManager.Users.ToList();
            return View(employees);
        }

        // Create Employee (GET)
        public IActionResult Create()
        {
            return View();
        }

        // Create Employee (POST)
```

```

[HttpPost]
[ValidateAntiForgeryToken]
public async Task<ActionResult> Create(ApplicationUser model)
{
    if (ModelState.IsValid)
    {
        var user = new ApplicationUser
        {
            UserName = model.UserName,
            Email = model.Email,
            Role = model.Role
        };

        var result = await _userManager.CreateAsync(user, model.Password);

        if (result.Succeeded)
        {
            await _userManager.AddToRoleAsync(user, model.Role);
            return RedirectToAction(nameof(Index));
        }

        foreach (var error in result.Errors)
        {
            ModelState.AddModelError(string.Empty, error.Description);
        }
    }

    return View(model);
}

// Delete Employee
[HttpPost]
[ValidateAntiForgeryToken]
public async Task<ActionResult> Delete(string id)
{
    var user = await _userManager.FindByIdAsync(id);
    if (user != null)
    {
        await _userManager.DeleteAsync(user);
    }

    return RedirectToAction(nameof(Index));
}
}
}

```

Program.cs

Program.cs

csharp

```

builder.Services.AddIdentity<ApplicationUser, IdentityRole>()
    .AddEntityFrameworkStores<ApplicationDbContext>()
    .AddDefaultTokenProviders();

```

ApplicationUser

```
bash

dotnet ef migrations add ExtendIdentityUser
dotnet ef database update
```

AspNetUsers

RegisterViewModel

AspNetUsers

RegisterViewModel

ApplicationUser

ApplicationUser

AspNetUsers

ApplicationUser

```
csharp

using Microsoft.AspNetCore.Identity;

namespace ETMS.Models
{
    public class ApplicationUser : IdentityUser
    {
        public string Role { get; set; }
    }
}
```

ApplicationDbContext

ApplicationUser

```
csharp

using ETMS.Models;
using Microsoft.AspNetCore.Identity.EntityFrameworkCore;
using Microsoft.EntityFrameworkCore;

namespace ETMS.Data
{
    public class ApplicationDbContext : IdentityDbContext<ApplicationUser>
    {
        public ApplicationDbContext(DbContextOptions<ApplicationDbContext> options) : base(options) { }
    }
}
```

```
}  
}
```

RegisterViewModel

RegisterViewModel

csharp

```
using ETMS.Data;  
using ETMS.Models;  
using Microsoft.AspNetCore.Identity;  
using Microsoft.AspNetCore.Mvc;  
using System.Linq;  
using System.Threading.Tasks;  
  
namespace ETMS.Controllers  
{  
    public class EmployeeManagementController : Controller  
    {  
        private readonly UserManager<ApplicationUser> _userManager;  
  
        public EmployeeManagementController(UserManager<ApplicationUser> userManager)  
        {  
            _userManager = userManager;  
        }  
  
        // Display All Employees  
        public IActionResult Index()  
        {  
            var employees = _userManager.Users.ToList();  
            return View(employees);  
        }  
  
        // GET: Create Employee  
        public IActionResult Create()  
        {  
            return View();  
        }  
  
        // POST: Create Employee  
        [HttpPost]  
        [ValidateAntiForgeryToken]  
        public async Task<IActionResult> Create(RegisterViewModel model)  
        {  
            if (ModelState.IsValid)  
            {  
                var user = new ApplicationUser  
                {  
                    UserName = model.UserName,  
                    Email = model.Email,  
                    Role = model.Role  
                };  
  
                var result = await _userManager.CreateAsync(user, model.Password);  
  
                if (result.Succeeded)
```

```

    {
        await _userManager.AddToRoleAsync(user, model.Role);
        return RedirectToAction(nameof(Index));
    }

    foreach (var error in result.Errors)
    {
        ModelState.AddModelError(string.Empty, error.Description);
    }
}

return View(model);
}

// GET: Edit Employee
public async Task<IActionResult> Edit(string id)
{
    var user = await _userManager.FindByIdAsync(id);
    if (user == null) return NotFound();

    var model = new RegisterViewModel
    {
        UserName = user.UserName,
        Email = user.Email,
        Role = user.Role
    };
    return View(model);
}

// POST: Edit Employee
[HttpPost]
[ValidateAntiForgeryToken]
public async Task<IActionResult> Edit(string id, RegisterViewModel model)
{
    var user = await _userManager.FindByIdAsync(id);
    if (user == null) return NotFound();

    if (ModelState.IsValid)
    {
        user.UserName = model.UserName;
        user.Email = model.Email;
        user.Role = model.Role;

        var result = await _userManager.UpdateAsync(user);

        if (result.Succeeded)
        {
            return RedirectToAction(nameof(Index));
        }

        foreach (var error in result.Errors)
        {
            ModelState.AddModelError(string.Empty, error.Description);
        }
    }
    return View(model);
}

// POST: Delete Employee
[HttpPost]
[ValidateAntiForgeryToken]
public async Task<IActionResult> Delete(string id)
{

```

```
var user = await _userManager.FindByIdAsync(id);
if (user == null) return NotFound();

await _userManager.DeleteAsync(user);
return RedirectToAction(nameof(Index));
    }
}
}
```

Program.cs

Program.cs

csharp

```
builder.Services.AddIdentity<ApplicationUser, IdentityRole>()
    .AddEntityFrameworkStores<ApplicationDbContext>()
    .AddDefaultTokenProviders();
```

bash

```
dotnet ef migrations add AddApplicationUser
dotnet ef database update
```

- RegisterViewModel
-
-

AspNetUsers

ApplicationUser

Index.cshtml

html


```
@model IEnumerable<ETMS.Models.ApplicationUser>

@{
    ViewData["Title"] = "Employee Management";
}

<div class="container mt-4">
    <h2>Employee Management</h2>

    <!-- Add New Employee Button -->
    <a asp-controller="EmployeeManagement" asp-action="Create" class="btn btn-primary mb-3">Add New Employee</a>

    <!-- Employee List Table -->
    <table class="table table-bordered table-striped">
        <thead class="thead-dark">
            <tr>
                <th>ID</th>
                <th>Username</th>
                <th>Email</th>
                <th>Role</th>
                <th>Actions</th>
            </tr>
        </thead>
        <tbody>
            @foreach (var employee in Model)
            {
                <tr>
                    <td>@employee.Id</td>
                    <td>@employee.UserName</td>
                    <td>@employee.Email</td>
                    <td>@employee.Role</td>
                    <td>
                        <!-- Edit Button -->
                        <a asp-controller="EmployeeManagement" asp-action="Edit" asp-route-id="@employee.Id" class="btn btn-warning btn-sm">Edit</a>

                        <!-- Delete Button -->
                        <form asp-controller="EmployeeManagement" asp-action="Delete" asp-route-id="@employee.Id" method="post" style="display:inline;">
                            @Html.AntiForgeryToken()
                            <button type="submit" class="btn btn-danger btn-sm" onclick="return confirm('Are you sure you want to delete this employee?');">Delete</button>
                        </form>
                    </td>
                </tr>
            }
        </tbody>
    </table>
</div>
```

Create.cshtml

html

@model ETMS.Models.RegisterViewModel

```
@{
    ViewData["Title"] = "Add Employee";
}

<div class="container mt-4">
    <h2>Add New Employee</h2>

    <form asp-controller="EmployeeManagement" asp-action="Create" method="post">
        @Html.AntiForgeryToken()

        <!-- Username -->
        <div class="form-group">
            <label asp-for="UserName">Username</label>
            <input asp-for="UserName" class="form-control" placeholder="Enter username" />
            <span asp-validation-for="UserName" class="text-danger"></span>
        </div>

        <!-- Email -->
        <div class="form-group">
            <label asp-for="Email">Email</label>
            <input asp-for="Email" class="form-control" placeholder="Enter email" />
            <span asp-validation-for="Email" class="text-danger"></span>
        </div>

        <!-- Password -->
        <div class="form-group">
            <label asp-for="Password">Password</label>
            <input asp-for="Password" type="password" class="form-control" placeholder="Enter password" />
            <span asp-validation-for="Password" class="text-danger"></span>
        </div>

        <!-- Confirm Password -->
        <div class="form-group">
            <label asp-for="ConfirmPassword">Confirm Password</label>
            <input asp-for="ConfirmPassword" type="password" class="form-control" placeholder="Confirm password" />
            <span asp-validation-for="ConfirmPassword" class="text-danger"></span>
        </div>

        <!-- Role -->
        <div class="form-group">
            <label asp-for="Role">Role</label>
            <select asp-for="Role" class="form-control">
                <option value="">-- Select Role --</option>
                <option value="Admin">Admin</option>
                <option value="Trainer">Trainer</option>
                <option value="Employee">Employee</option>
            </select>
            <span asp-validation-for="Role" class="text-danger"></span>
        </div>

        <!-- Submit Button -->
        <button type="submit" class="btn btn-success">Add Employee</button>

        <!-- Back Link -->
        <a asp-action="Index" class="btn btn-secondary">Back to List</a>
    </form>
</div>
```

Edit.cshtml

```
html

@model ETMS.Models.RegisterViewModel

@{
    ViewData["Title"] = "Edit Employee";
}

<div class="container mt-4">
    <h2>Edit Employee</h2>

    <form asp-controller="EmployeeManagement" asp-action="Edit" method="post">
        @Html.AntiForgeryToken()

        <!-- Hidden ID -->
        <input type="hidden" asp-for="UserName" />

        <!-- Username -->
        <div class="form-group">
            <label asp-for="UserName">Username</label>
            <input asp-for="UserName" class="form-control" readonly />
        </div>

        <!-- Email -->
        <div class="form-group">
            <label asp-for="Email">Email</label>
            <input asp-for="Email" class="form-control" />
            <span asp-validation-for="Email" class="text-danger"></span>
        </div>

        <!-- Role -->
        <div class="form-group">
            <label asp-for="Role">Role</label>
            <select asp-for="Role" class="form-control">
                <option value="Admin">Admin</option>
                <option value="Trainer">Trainer</option>
                <option value="Employee">Employee</option>
            </select>
            <span asp-validation-for="Role" class="text-danger"></span>
        </div>

        <!-- Submit Button -->
        <button type="submit" class="btn btn-primary">Update Employee</button>

        <!-- Back Link -->
        <a asp-action="Index" class="btn btn-secondary">Back to List</a>
    </form>
</div>
```

- ApplicationUser ApplicationDbContext
- EmployeeManagementController UserManager<ApplicationUser>
-

RegisterViewModel RegisterViewModel

Id

RegisterViewModel DbSet ApplicationDbContext

ApplicationUser
RegisterViewModel

ApplicationUser

```
csharp
using Microsoft.AspNetCore.Identity;

namespace ETMS.Models
{
    public class ApplicationUser : IdentityUser
    {
        public string Role { get; set; }
    }
}
```

ApplicationDbContext

ApplicationUser

RegisterViewModel

csharp

```
using Microsoft.AspNetCore.Identity.EntityFrameworkCore;
using Microsoft.EntityFrameworkCore;

namespace ETMS.Models
{
    public class ApplicationDbContext : IdentityDbContext<ApplicationUser>
    {
        public ApplicationDbContext(DbContextOptions<ApplicationDbContext> options)
            : base(options) { }

        protected override void OnModelCreating(ModelBuilder builder)
        {
            base.OnModelCreating(builder);
        }
    }
}
```

ApplicationUser

RegisterViewModel

csharp

```
using ETMS.Models;
using Microsoft.AspNetCore.Identity;
using Microsoft.AspNetCore.Mvc;
using System.Threading.Tasks;

namespace ETMS.Controllers
{
    public class EmployeeManagementController : Controller
    {
        private readonly UserManager<ApplicationUser> _userManager;

        public EmployeeManagementController(UserManager<ApplicationUser> userManager)
        {
            _userManager = userManager;
        }

        public IActionResult Create() => View();

        [HttpPost]
        [ValidateAntiForgeryToken]
        public async Task<IActionResult> Create(RegisterViewModel model)
        {
            if (!ModelState.IsValid) return View(model);

            var user = new ApplicationUser
            {
                UserName = model.UserName,
```

```
        Email = model.Email,
        Role = model.Role
    };

    var result = await _userManager.CreateAsync(user, model.Password);

    if (result.Succeeded)
        return RedirectToAction("Index");

    foreach (var error in result.Errors)
        ModelState.AddModelError(string.Empty, error.Description);

    return View(model);
}
}
```

```
bash

dotnet ef migrations add UpdateApplicationUser
dotnet ef database update
```

RegisterViewModel

ApplicationUser

DbContext

ApplicationUser

RegisterViewModel

ApplicationDbContext

RegisterViewModel

DbSet

RegisterViewModel

RegisterViewModel

DbContext

ApplicationDbContext

RegisterViewModel

DbSet

ApplicationDbContext

ApplicationDbContext

RegisterViewModel

ApplicationDbContext

csharp

```
using Microsoft.AspNetCore.Identity.EntityFrameworkCore;
using Microsoft.EntityFrameworkCore;
using ETMS.Models;

namespace ETMS.Data
{
    public class ApplicationDbContext : IdentityDbContext<ApplicationUser>
    {
        public ApplicationDbContext(DbContextOptions<ApplicationDbContext> options)
            : base(options)
        {
        }

        // Ensure ONLY entity models are included here
        public DbSet<ApplicationUser> Users { get; set; }
    }
}
```

RegisterViewModel

ApplicationDbContext

csharp

```
public DbSet<RegisterViewModel> Employees { get; set; } // REMOVE THIS!
```

RegisterViewModel

DbContext

- RegisterViewModel
-

RegisterViewModel

RegisterViewModel

csharp

```
using System.ComponentModel.DataAnnotations;

namespace ETMS.Models
{
    public class RegisterViewModel
    {
        [Required]
        public string UserName { get; set; }

        [Required]
        [EmailAddress]
        public string Email { get; set; }

        [Required]
        [DataType(DataType.Password)]
        public string Password { get; set; }

        [Required]
        [Compare("Password")]
        public string ConfirmPassword { get; set; }

        [Required]
        public string Role { get; set; }
    }
}
```

RegisterViewModel

bash

dotnet ef migrations remove

bash

dotnet ef migrations add InitialCreate
dotnet ef database update

- `ApplicationUser`
-

xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx

4799338a-5052-4c6d-8cbe-0207be0a7e76

- `.NewGuid()`
- `UUID()`

`asp-action="Index"`

`Index`

csharp

```
public IActionResult Index()
{
    var employees = _userManager.Users.ToList();
    return View(employees);
}
```

```
html

<a asp-controller="EmployeeManagement" asp-action="Index" class="btn btn-secondary">Back to List</a>
```

```
html

<a asp-action="Index" class="btn btn-secondary">Back to List</a>
```

```
html

<h2>Edit Employee</h2>

<form asp-action="Edit" method="post">
    <!-- Form fields here -->
    <button type="submit" class="btn btn-primary">Save</button>

    <!-- Back Button -->
    <a asp-controller="EmployeeManagement" asp-action="Index" class="btn btn-secondary">Back to List</a>
</form>
```

EmployeeManagementController

Index

Program.cs

Startup.cs

```
csharp

app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Home}/{action=Index}/{id?}");
```

ModelState

RegisterViewModel

id

csharp

```
if (!ModelState.IsValid)
{
    foreach (var modelError in ModelState.Values.SelectMany(v => v.Errors))
    {
        Console.WriteLine($"Error: {modelError.ErrorMessage}");
    }
    return View(model);
}
```

RegisterViewModel

csharp

```
public class RegisterViewModel
{
    public string UserName { get; set; }

    [EmailAddress(ErrorMessage = "Invalid Email Address")]
    public string Email { get; set; }

    public string Role { get; set; }

    [Required(ErrorMessage = "Password is required.")]
```

```
public string Password { get; set; }  
}
```

html

```
<form asp-action="Edit" method="post">  
  <input type="hidden" asp-for="Id" />  
  
  <div class="form-group">  
    <label asp-for="UserName"></label>  
    <input asp-for="UserName" class="form-control" />  
    <span asp-validation-for="UserName" class="text-danger"></span>  
  </div>  
  
  <div class="form-group">  
    <label asp-for="Email"></label>  
    <input asp-for="Email" class="form-control" />  
    <span asp-validation-for="Email" class="text-danger"></span>  
  </div>  
  
  <div class="form-group">  
    <label asp-for="Role"></label>  
    <input asp-for="Role" class="form-control" />  
    <span asp-validation-for="Role" class="text-danger"></span>  
  </div>  
  
  <button type="submit" class="btn btn-primary">Update Employee</button>  
</form>
```

csharp

```
[HttpPost]  
[ValidateAntiForgeryToken]  
public async Task<ActionResult> Edit(string id, RegisterViewModel model)  
{  
    if (!ModelState.IsValid)  
    {  
        foreach (var error in ModelState.Values.SelectMany(v => v.Errors))  
        {  
            Console.WriteLine($"Error: {error.ErrorMessage}");  
        }  
        return View(model);  
    }  
  
    var user = await _userManager.FindByIdAsync(id);  
    if (user == null) return NotFound();  
  
    user.UserName = model.UserName;  
    user.Email = model.Email;  
    user.Role = model.Role;  
  
    var result = await _userManager.UpdateAsync(user);  
    if (result.Succeeded)
```

```
{
    return RedirectToAction(nameof(Index));
}

foreach (var error in result.Errors)
{
    ModelState.AddModelError(string.Empty, error.Description);
}

return View(model);
}
```

ModelState

RegisterViewModel does not contain a definition for 'Id'

RegisterViewModel

RegisterViewModel

Id

Id

Id

RegisterViewModel

csharp

```
public class RegisterViewModel
{
    public string Id { get; set; } // <-- Add this line for user identification.

    public string UserName { get; set; }

    [EmailAddress(ErrorMessage = "Invalid Email Address")]
    public string Email { get; set; }

    public string Role { get; set; }

    [Required(ErrorMessage = "Password is required.")]
```

```
public string Password { get; set; }  
}
```

Edit

Id

csharp

```
// GET: Edit Employee  
public async Task<ActionResult> Edit(string id)  
{  
    var user = await _userManager.FindByIdAsync(id);  
    if (user == null) return NotFound();  
  
    var model = new RegisterViewModel  
    {  
        Id = user.Id, // <-- Pass the Id to the view.  
        UserName = user.UserName,  
        Email = user.Email,  
        Role = user.Role  
    };  
    return View(model);  
}
```

csharp

```
// POST: Edit Employee  
[HttpPost]  
[ValidateAntiForgeryToken]  
public async Task<ActionResult> Edit(string id, RegisterViewModel model)  
{  
    if (!ModelState.IsValid)  
    {  
        foreach (var error in ModelState.Values.SelectMany(v => v.Errors))  
        {  
            Console.WriteLine($"Error: {error.ErrorMessage}");  
        }  
        return View(model);  
    }  
  
    var user = await _userManager.FindByIdAsync(id);  
    if (user == null) return NotFound();  
  
    user.UserName = model.UserName;  
    user.Email = model.Email;  
    user.Role = model.Role;  
  
    var result = await _userManager.UpdateAsync(user);  
    if (result.Succeeded)  
    {  
        return RedirectToAction(nameof(Index));  
    }  
  
    foreach (var error in result.Errors)  
    {
```

```
ModelState.AddModelError(string.Empty, error.Description);
}

return View(model);
}
```

Edit.cshtml

```
html

<form asp-action="Edit" method="post">
  <input type="hidden" asp-for="Id" /> <!-- Ensure the Id is here -->

  <div class="form-group">
    <label asp-for="UserName"></label>
    <input asp-for="UserName" class="form-control" />
    <span asp-validation-for="UserName" class="text-danger"></span>
  </div>

  <div class="form-group">
    <label asp-for="Email"></label>
    <input asp-for="Email" class="form-control" />
    <span asp-validation-for="Email" class="text-danger"></span>
  </div>

  <div class="form-group">
    <label asp-for="Role"></label>
    <input asp-for="Role" class="form-control" />
    <span asp-validation-for="Role" class="text-danger"></span>
  </div>

  <button type="submit" class="btn btn-primary">Update Employee</button>
</form>
```

Ctrl + Shift + B

Id

Id